NAME: K. SWARNA VARSHINI

REG.NO: 192324081

COURSE CODE: CSA0920

GITHUB LINK:

https://github.com/kswarna1/CSA0920-JAVA.git

Q1–Building Text Under Load: String, StringBuilder or StringBuffer?

Aim:

To implement and compare String, StringBuilder and StringBuffer in a railway e-ticket generation system by measuring execution time, estimating object creation and analysing memory usage under different workloads.

Theory:

String: String is immutable. Whenever a new value is assigned through concatenation, a new String object is created. Repeated concatenation using + can reduce performance.

StringBuilder: StringBuilder is mutable. It modifies the existing character buffer using the append() method. It is suitable for repeated string operations in single-threaded tasks.

StringBuffer: StringBuffer is mutable and synchronized. It supports thread-safe individual operations and is useful when multiple threads modify a shared buffer.

Feature	**String**	**StringBuilder**	**StringBuffer**
Mutable	No	Yes	Yes
Synchronization	Not required	No	Yes
Performance for repeated append	Lower	Higher	Generally lower than StringBuilder
Suitable for	Fixed text	Individual tickets	Shared log

Workloads:

- Workload 1: 1 ticket with 6 fields.
- Workload 2: 10,000 tickets with 50 fields.
- Workload 3: 2,000 tickets with 100 fields.

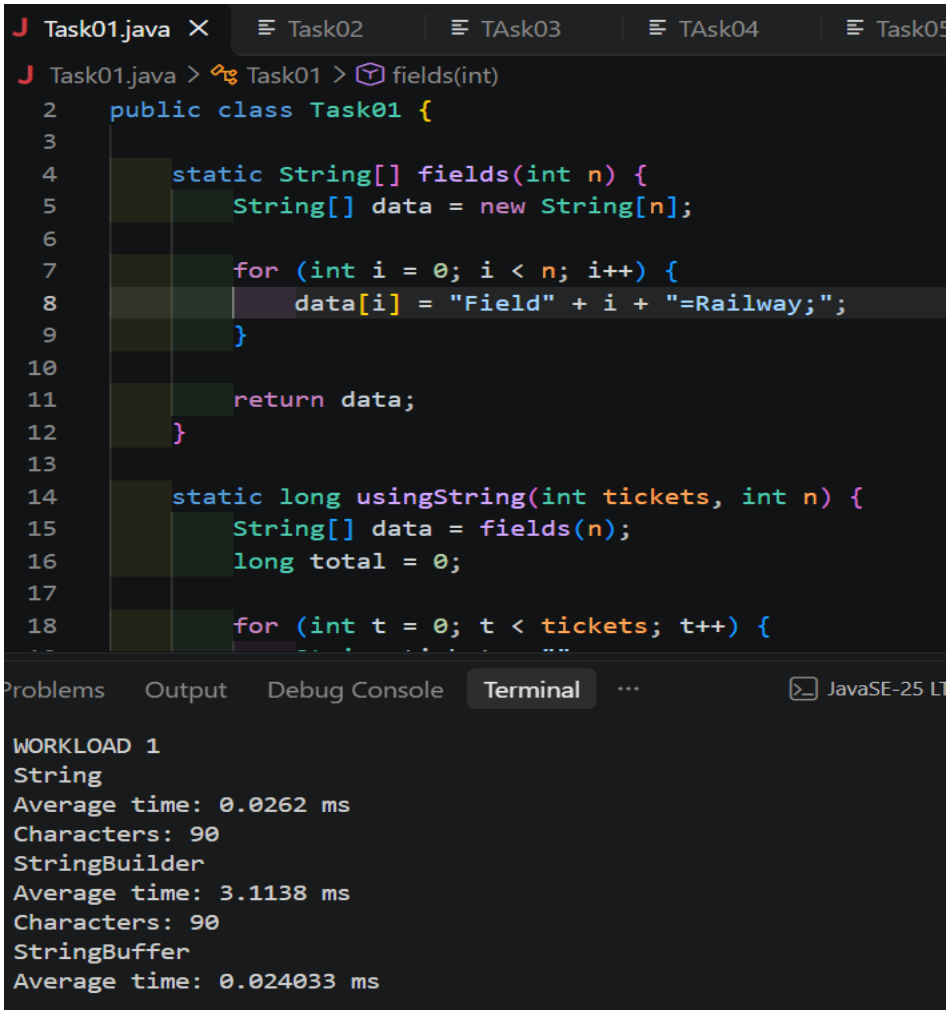
Java Program:

```
public class Main {
    static String[] fields(int n) {
        String[] data = new String[n];
        for (int i = 0; i < n; i++) {
            data[i] = "Field" + i + "=Railway;";
        }
        return data;
    }
    static long usingString(int tickets, int n) {
        String[] data = fields(n);
        long total = 0;
        for (int t = 0; t < tickets; t++) {
            String ticket = "";
            for (int i = 0; i < n; i++) {
                ticket = ticket + data[i];
            }
            total += ticket.length();
        }
        return total;
    }
    static long usingBuilder(int tickets, int n) {
        String[] data = fields(n);
        long total = 0;
```

```
for (int t = 0; t < tickets; t++) {
        StringBuilder ticket = new StringBuilder();
        for (int i = 0; i < n; i++) {
            ticket.append(data[i]);
        }
        total += ticket.length();
    }
    return total;
  }
  static long usingBuffer(int tickets, int n) {
    String[] data = fields(n);
    long total = 0;
    for (int t = 0; t < tickets; t++) {
        StringBuffer ticket = new StringBuffer();
        for (int i = 0; i < n; i++) {
            ticket.append(data[i]);
        }
        total += ticket.length();
    }
    return total;
  }
  static void test(String name, int tickets, int n,
                   java.util.function.LongSupplier task) {
```

```
    long totalTime = 0;
    long result = 0;
    task.getAsLong(); // Warm-up
    for (int i = 0; i < 3; i++) {
      long start = System.nanoTime();
      result = task.getAsLong();
      long end = System.nanoTime();
      totalTime += end - start;
    }
    System.out.println(name);
    System.out.println("Average time: "
      + totalTime / 3 / 1000000.0 + " ms");
    System.out.println("Characters: " + result);
  }
  public static void main(String[] args) {
    System.out.println("WORKLOAD 1");
    test("String", 1, 6, () -> usingString(1, 6));
    test("StringBuilder", 1, 6, () -> usingBuilder(1, 6));
    test("StringBuffer", 1, 6, () -> usingBuffer(1, 6));
    System.out.println("\nWORKLOAD 2");
    test("String", 10000, 50,
      () -> usingString(10000, 50));
    test("StringBuilder", 10000, 50,
      () -> usingBuilder(10000, 50));
```

```
        test("StringBuffer", 10000, 50,
            () -> usingBuffer(10000, 50));
        System.out.println("\nWORKLOAD 3");
        test("String", 2000, 100,
            () -> usingString(2000, 100));
        test("StringBuilder", 2000, 100,
            () -> usingBuilder(2000, 100));
        test("StringBuffer", 2000, 100,
            () -> usingBuffer(2000, 100));
    }
}
```



Q2– When Is Array List an Appropriate Engineering Choice?

Aim

To compare ArrayList and LinkedList based on insertion, deletion, searching, random access and iteration operations in a music playlist application, and to evaluate ArrayDeque for an Up Next queue.

Theory

ArrayList:

- Uses a dynamic array internally.
- Allows direct access to elements using indexes.
- Insertion and deletion in the middle require shifting elements.

LinkedList:

- Uses nodes connected through references.
- Insertion and deletion at known positions are efficient.
- Random access requires sequential traversal.

ArrayDeque:

- Uses a resizable array.
- Supports insertion and deletion at both ends.
- Suitable for implementing queues and stacks.

Java Program

```
import java.util.*;

public class Main {

    static void test(int n) {

        ArrayList<Integer> a = new ArrayList<>();

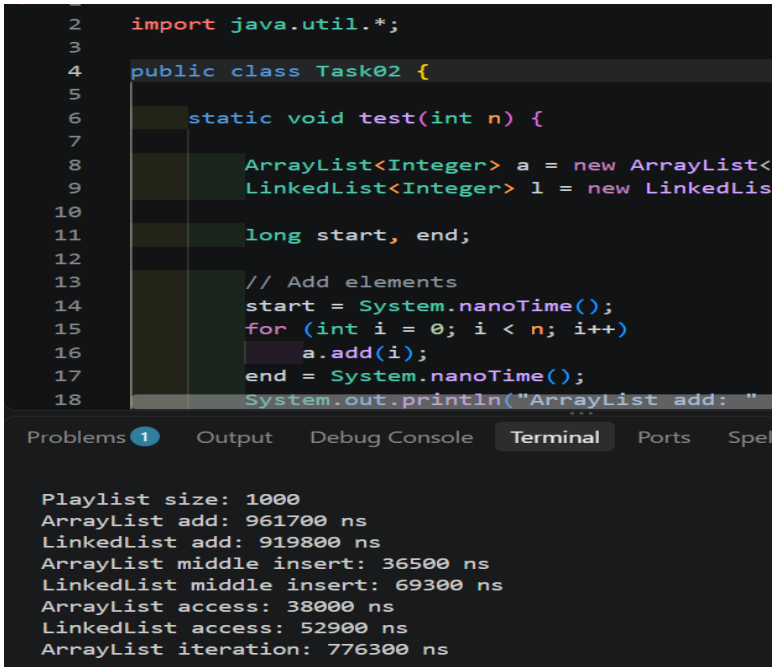
        LinkedList<Integer> l = new LinkedList<>();

        long start, end;
```

```
// Add elements
start = System.nanoTime();
for (int i = 0; i < n; i++)
    a.add(i);
end = System.nanoTime();
System.out.println("ArrayList add: " + (end-start) + " ns");
start = System.nanoTime();
for (int i = 0; i < n; i++)
    l.add(i);
end = System.nanoTime();
System.out.println("LinkedList add: " + (end-start) + " ns");
// Insert in middle
start = System.nanoTime();
a.add(n / 2, -1);
end = System.nanoTime();
System.out.println("ArrayList middle insert: " + (end-start) + " ns");
start = System.nanoTime();
l.add(n / 2, -1);
end = System.nanoTime();
System.out.println("LinkedList middle insert: " + (end-start) + " ns");
// Random access
start = System.nanoTime();
int x = a.get(n / 2);
end = System.nanoTime();
```

```
    System.out.println("ArrayList access: " + (end-start) + " ns");
    start = System.nanoTime();
    x = l.get(n / 2);
    end = System.nanoTime();
    System.out.println("LinkedList access: " + (end-start) + " ns");
    // Iteration
    start = System.nanoTime();
    for (int value : a) x += value;
    end = System.nanoTime();
    System.out.println("ArrayList iteration: " + (end-start) + " ns");
    start = System.nanoTime();
    for (int value : l) x += value;
    end = System.nanoTime();
    System.out.println("LinkedList iteration: " + (end-start) + " ns");
    // Remove front
    start = System.nanoTime();
    a.remove(0);
    end = System.nanoTime();
    System.out.println("ArrayList front remove: " + (end-start) + " ns");
    start = System.nanoTime();
    l.remove(0);
    end = System.nanoTime();
    System.out.println("LinkedList front remove: " + (end-start) + " ns");
}
```

```
    public static void main(String[] args) {
        int[] sizes = {1000, 10000, 50000};
        for (int n : sizes) {
            System.out.println("\nPlaylist size: " + n);
            test(n);
        }
        // Up Next queue using ArrayDeque
        ArrayDeque<String> queue = new ArrayDeque<>();
        queue.addLast("Song A");
        queue.addLast("Song B");
        queue.addFirst("Song C");
        System.out.println("\nUp Next Queue: " + queue);
        System.out.println("Playing: " + queue.removeFirst());
    }
}
```



Q3–Engineer the hash Code–equals Contract

Aim

To investigate the relationship between equals() and hashCode() methods in Java and understand how HashSet identifies duplicate student attendance records.

Theory

equals():

- Compares two objects to determine whether they are logically equal.
- In this program, students with the same registration number are considered equal.

hashCode():

- Returns an integer hash value for an object.
- HashSet uses this value to identify the bucket where an object should be stored.

HashSet:

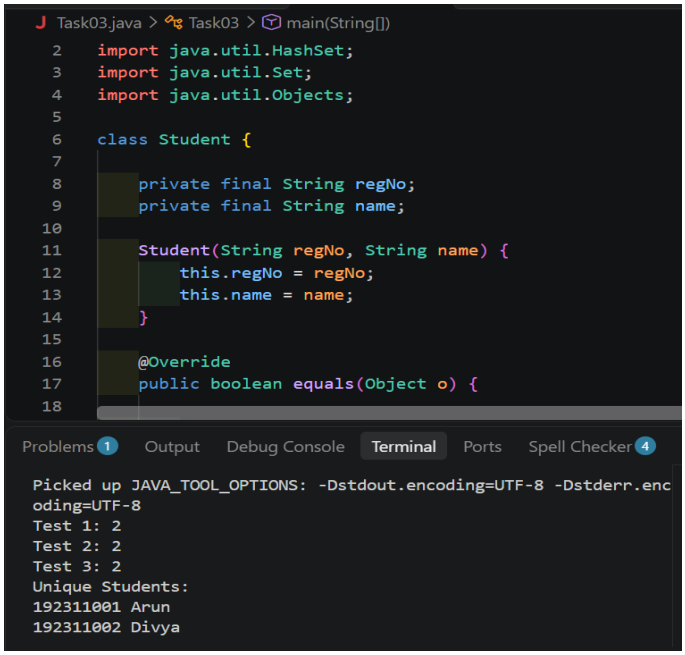
- Stores unique elements.
- Uses hashCode() and equals() to identify duplicates.
- Does not maintain insertion order.

Java Program

```
import java.util.HashSet;
import java.util.Set;
import java.util.Objects;
class Student {
    private final String regNo;
    private final String name;
    Student(String regNo, String name) {
        this.regNo = regNo;
        this.name = name;
```

```
    }
    @Override
    public boolean equals(Object o) {
        if (this == o)
            return true;
        if (!(o instanceof Student))
            return false;
        Student s = (Student) o;
        return regNo.equals(s.regNo);
    }
    // Correct hashCode implementation
    @Override
    public int hashCode() {
        return Objects.hash(regNo);
    }
    @Override
    public String toString() {
        return regNo + " " + name;
    }
}
public class Attendance {
    public static void main(String[] args) {
        Set<Student> present = new HashSet<>();
 // Test 1
```

```
        present.add(new Student("192311001", "Arun"));
        present.add(new Student("192311002", "Divya"));
        System.out.println("Test 1: " + present.size());
        // Test 2
        present.add(new Student("192311001", "Arun"));
        System.out.println("Test 2: " + present.size());
        // Test 3
        present.add(new Student("192311001", "Kumar"));
        System.out.println("Test 3: " + present.size());
        System.out.println("Unique Students:");
        for (Student s : present) {
            System.out.println(s);
        }
    }
}
```



Q4–Choosing a Map for a Pharmacy Inventory

Aim

To implement and compare HashMap, TreeMap and LinkedHashMap for managing hospital pharmacy inventory and identify the suitable map for fast stock lookup and ordered reports.

Theory

Hash Map:

- Stores key-value pairs.
- Uses hashing for efficient searching.
- Does not guarantee insertion or sorted order.
- Average put() and get() complexity is O(1).

Tree Map:

- Stores keys in sorted order.
- Uses a Red-Black Tree internally.
- Provides O(log n) insertion, searching and deletion.
- Suitable for reports in medicine-code order.

Linked HashMap:

- Maintains insertion order.
- Provides average O(1) lookup.
- Suitable for reports in the order medicines were added.

Java Program

```
import java.util.*;

public class Pharmacy {

    public static void main(String[] args) {

        HashMap<String, Integer> hash =

            new HashMap<>();
```

```
TreeMap<String, Integer> tree =
    new TreeMap<>();
LinkedHashMap<String, Integer> linked =
    new LinkedHashMap<>();
String[] codes = {
    "M103", "M101", "M105", "M102", "M104"
};
int[] stocks = {50, 100, 30, 80, 60};
for (int i = 0; i < codes.length; i++) {
    hash.put(codes[i], stocks[i]);
    tree.put(codes[i], stocks[i]);
    linked.put(codes[i], stocks[i]);
}
// HashMap lookup
System.out.println("Stock Lookup:");
System.out.println("M101: " + hash.get("M101"));
// Code order report
System.out.println("\nTreeMap - Code Order:");
for (Map.Entry<String, Integer> e : tree.entrySet()) {
    System.out.println(e.getKey() + " = " + e.getValue());
}
// Insertion order report
System.out.println("\nLinkedHashMap - Insertion Order:");
for (Map.Entry<String, Integer> e : linked.entrySet()) {
```

```
    System.out.println(e.getKey() + " = " + e.getValue());
}
// Performance test
int n = 100000;
for (int i = 0; i < n; i++) {
    hash.put("X" + i, i);
    tree.put("X" + i, i);
    linked.put("X" + i, i);
}
long start, end;
start = System.nanoTime();
for (int i = 0; i < n; i++) {
    hash.get("X" + i);
}
end = System.nanoTime();
System.out.println("\nHashMap lookup: "
    + (end - start) + " ns");
start = System.nanoTime();
for (int i = 0; i < n; i++) {
    tree.get("X" + i);
}
end = System.nanoTime();
System.out.println("TreeMap lookup: "
    + (end - start) + " ns");
```

```
        start = System.nanoTime();
        for (int i = 0; i < n; i++) {
            linked.get("X" + i);
        }
        end = System.nanoTime();
        System.out.println("LinkedHashMap lookup: "
            + (end - start) + " ns");
    }
}
```

Task04.java > Task04

```
import java.util.*;

public class Task04 {

    Run | Debug
    public static void main(String[] args) {

        HashMap<String, Integer> hash =
            new HashMap<>();

        TreeMap<String, Integer> tree =
            new TreeMap<>();

        LinkedHashMap<String, Integer> linked =
            new LinkedHashMap<>();

        String[] codes = {
```

roblems 1 | Output | Debug Console | Terminal | Ports | Spell Che

```
TreeMap - Code Order:
M101 = 100
M102 = 80
M103 = 50
M104 = 60
M105 = 30

LinkedHashMap - Insertion Order:
```

Q5–Derive a Generic Ranking Component from a Principle

Aim

To develop a generic leaderboard using Java Generics and Comparator to rank sports teams based on points, goal difference and team name, and to verify the correctness of the ranking order.

Theory

Comparable:

- Comparable is an interface used to define the natural ordering of objects.
- It contains the compareTo() method.
- A class implementing Comparable defines its own default sorting order.

Comparator:

- Comparator is an interface used to define custom sorting rules.
- It contains the compare() method.
- Multiple Comparator objects can be created for different sorting requirements.

Generics:

- Generics allow classes and methods to work with different data types.
- They provide type safety and code reusability.

Wildcard:

- List<? extends T> allows a method to accept a list containing T or its subclasses.

Ranking rules

1. Higher points first.
2. Higher goal difference first.
3. Team name in alphabetical order.

Java Program

```
import java.util.ArrayList;
import java.util.Comparator;
import java.util.List;
class Team {
    final String name;
    final int points;
    final int goalDiff;
    Team(String name, int points, int goalDiff) {
        this.name = name;
        this.points = points;
        this.goalDiff = goalDiff;
    }
    @Override
    public String toString() {
        return name + "(" + points + "," + goalDiff + ")";
    }
}
class Leaderboard<T> {
    private final List<T> items = new ArrayList<>();
    private final Comparator<? super T> order;
    Leaderboard(Comparator<? super T> order) {
        this.order = order;
    }
```

```
    void add(T item) {
        items.add(item);
    }
    List<T> ranking() {
        List<T> result = new ArrayList<>(items);
        result.sort(order);
        return result;
    }
}
public class LeaderboardDemo {
    public static void main(String[] args) {
        Comparator<Team> byRank =
            Comparator.comparingInt((Team t) -> t.points)
            .reversed()
            .thenComparing(
                Comparator.comparingInt(
                    (Team t) -> t.goalDiff
                ).reversed()
            )
            .thenComparing(t -> t.name);
        Leaderboard<Team> board =
            new Leaderboard<>(byRank);
        board.add(new Team("Panthers", 9, 4));
        board.add(new Team("Eagles", 9, 6));
```

```
        board.add(new Team("Cobras", 6, 1));
        board.add(new Team("Bisons", 9, 4));
        System.out.println(board.ranking());
    }
}
```

```
24  class Leaderboard<T> {
37      List<T> ranking() {
38
39          List<T> result = new ArrayList<>(items);
40
41          result.sort(order);
42
43          return result;
44      }
45  }
46
47  public class Task05 {
48
        Run | Debug
49      public static void main(String[] args) {
50
51          Comparator<Team> byRank =
```

```
Problems 1   Output   Debug Console   Terminal   Ports   Spell Checker

Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.
oding=UTF-8

D:\JAVA\CSA09-192324081-Assignment2>java Task05
Picked up JAVA_TOOL_OPTIONS: -Dstdout.encoding=UTF-8 -Dstderr.
oding=UTF-8
[Eagles(9,6), Bisons(9,4), Panthers(9,4), Cobras(6,1)]
```

Q6–Priority, Order and Fairness in a Queue

Aim

To implement a hospital emergency triage system using Java PriorityQueue and Comparator, where patients with higher severity are treated first and patients with equal severity are treated according to arrival order.

Theory

Queue:

- Queue follows FIFO (First In, First Out).

- The patient who arrives first is treated first.
- A normal queue does not consider patient severity.

PriorityQueue:

- PriorityQueue processes elements according to priority.
- It uses a heap data structure internally.
- The patient with the highest priority is removed first.

Comparator:

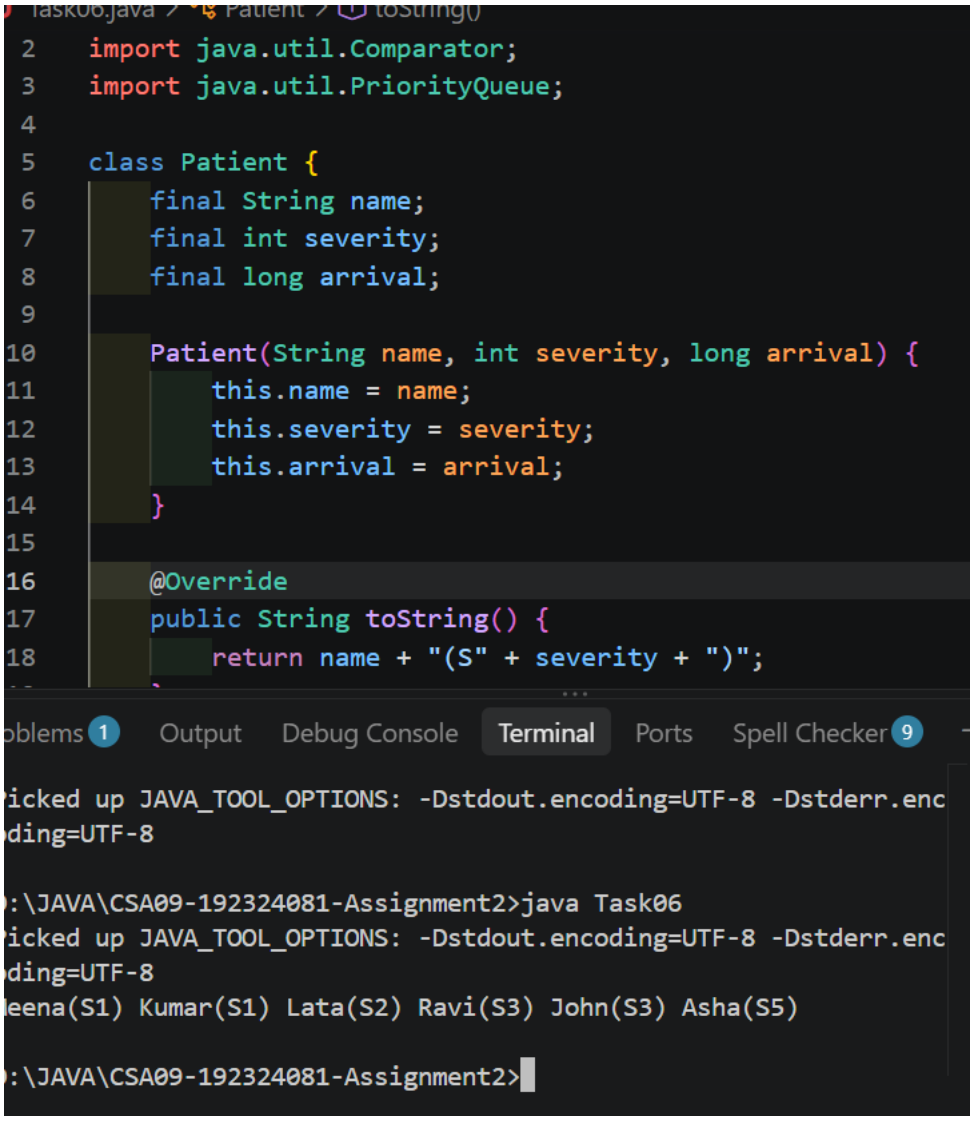
- Comparator defines the priority order.
- Severity 1 is treated as the highest priority.
- Arrival time is used to maintain fairness among patients with equal severity.

Java Program

```
import java.util.Comparator;
import java.util.PriorityQueue;
class Patient {
    final String name;
    final int severity;
    final long arrival;
    Patient(String name, int severity, long arrival) {
        this.name = name;
        this.severity = severity;
        this.arrival = arrival;
    }
    @Override
    public String toString() {
```

```
return name + "(S" + severity + ")";
    }
}
public class Task06 {
    private long counter = 0;
    PriorityQueue<Patient> queue =
        new PriorityQueue<>(
            Comparator.comparingInt((Patient p) -> p.severity)
                .thenComparingLong(p -> p.arrival)
        );
    void admit(String name, int severity) {
        queue.add(new Patient(name, severity, counter++));
    }
    Patient next() {
        return queue.poll();
    }
    public static void main(String[] args) {
        Task06 er = new Task06();
        er.admit("Ravi", 3);
        er.admit("Meena", 1);
        er.admit("John", 3);
        er.admit("Asha", 5);
        er.admit("Kumar", 1);
        er.admit("Lata", 2)
```

```
        StringBuilder order = new StringBuilder();
        Patient p;
        while ((p = er.next()) != null) {
            order.append(p).append(" ");
        }
        System.out.println(order.toString().trim());
    }
}
```



Q7–Diagnose, Redesign and Validate

Aim

To identify the performance limitation of ArrayList during repeated middle insertions in a music playlist and redesign the solution using a Gap Buffer to reduce repeated element shifting.

Theory

Original solution – ArrayList

ArrayList uses a dynamic array to store elements.

Advantages:

- Fast random access.
- Easy iteration.
- Efficient insertion at the end.

Limitation:

When a song is inserted in the middle, all subsequent elements must be shifted to make space.

Java Program – Original vs Redesigned Solution

```
import java.util.ArrayList;

class GapBuffer {
    String[] data;
    int gapStart, gapEnd;
    GapBuffer(int n) {
        data = new String[n + 5100];
        for (int i = 0; i < n; i++) {
            data[i] = "Song" + i;
        }
        gapStart = n;
        gapEnd = data.length;
```

```
}
void moveGap(int index) {
    while (gapStart > index) {
        data[--gapEnd] = data[--gapStart];
        data[gapStart] = null;
    }
    while (gapStart < index) {
        data[gapStart++] = data[gapEnd];
        data[gapEnd++] = null;
    }
}
void insert(int index, String song) {
    moveGap(index);
    data[gapStart++] = song;
}
int size() {
    return data.length - (gapEnd - gapStart);
}
String get(int index) {
    if (index < gapStart) {
        return data[index];
    }
    return data[index + (gapEnd - gapStart)];
}
```

```
}
public class Task07 {
    public static void main(String[] args) {
        int n = 10000;
        int insertions = 5000;
        int middle = n / 2;
        ArrayList<String> list = new ArrayList<>();
        GapBuffer buffer = new GapBuffer(n);
        for (int i = 0; i < n; i++) {
            list.add("Song" + i);
        }
        long start = System.nanoTime();
        for (int i = 0; i < insertions; i++) {
            list.add(middle, "NewSong" + i);
        }
        long arrayTime = System.nanoTime() - start;
        start = System.nanoTime();
        for (int i = 0; i < insertions; i++) {
            buffer.insert(middle, "NewSong" + i);
        }
        long gapTime = System.nanoTime() - start;
        System.out.println("TASK 07: GAP BUFFER COMPARISON");
        System.out.println("ArrayList Size: " + list.size());
        System.out.println("GapBuffer Size: " + buffer.size());
```

```
        System.out.println("ArrayList Middle: " + list.get(middle));
        System.out.println("GapBuffer Middle: " + buffer.get(middle));
        System.out.println("ArrayList Time: " + arrayTime + " ns");
        System.out.println("GapBuffer Time: " + gapTime + " ns");
    }
}
```

